

Reading the Residual Stream to Stop Thinking: Calibrated Convergence Probes for Training-Free Test-Time Compute Control

Anonymous Submission

Abstract

Reasoning language models spend most of their test-time compute after they have already determined their answer. We show that this “answer convergence” is linearly decodable from the residual stream of DeepSeek-R1-Distill models at both 1.5B and 7B scale (AUC 0.882 / 0.854), that the signal survives a strict position-controlled baseline within every position stratum (margin 0.147 / 0.246), and that it costs one matrix-vector product on activations the model has already computed—no auxiliary decode, no training, no weight updates. We convert this sensor into a training-free controller that decides EXIT / CONTINUE at reasoning-chunk boundaries, and evaluate it under a pre-registered protocol with dev-only tuning, single-shot test evaluation, and problem-level cluster bootstrap. The controller removes 52.4% (1.5B) and 58.3% (7B) of thinking tokens on pooled MATH-500 + GSM8K, at an accuracy cost of -0.039 [$-0.054, -0.024$] and -0.073 [$-0.091, -0.056$] respectively. At 7B it dominates a compute-matched static budget (27.1% fewer tokens at equal-or-better accuracy), and on GPQA-Diamond it *exceeds* unconstrained reasoning at both scales (+0.074, +0.049) while using far less compute. An operating point selected on the 1.5B transfers to the 7B with no retuning. We additionally report two results we believe generalize beyond this system. First, a causal steering channel built from the same representations fails decisively, and we characterize why. Second, and most consequentially for probe-based control in general, a probe that is well-calibrated on-policy (ECE ≈ 0.02) becomes badly miscalibrated under its own intervention: realized accuracy is flat at ≈ 0.69 across every claimed-confidence bin from 0.75 to 0.97. On-policy calibration is not evidence of calibration under control.

1 Introduction

Reasoning-tuned language models produce long chains of thought whose length is only loosely coupled to problem difficulty. A substantial fraction of that computation is spent after the model’s answer is, in effect, already fixed: the trailing tokens re-derive, re-verify, and restate. If a system could tell *when* the answer has converged, it could

stop early and recover that compute.

Existing approaches to this problem largely infer convergence from behavior: they decode a trial answer and check agreement [1, 2], monitor specialized neurons [3], or train opaque classifiers on embeddings [4]. Behavioral probes pay extra decoding; neuron heuristics and black-box classifiers are hard to audit and offer little insight into what is being measured.

We take a representational route. We train a linear probe on the residual stream to predict a well-defined target—whether truncating the chain *now* and forcing an answer would yield the same answer the model eventually produces—and use it as a sensor inside a decoding loop. The probe reads states the forward pass already produced, so the sensor is essentially free, and because it is linear and calibrated it can be inspected, ablated, and compared against position-only baselines.

Contributions.

1. **Convergence is linearly and calibratedly decodable, beyond position.** At both scales the convergence probe clears a pre-registered position-controlled gate within every position stratum, peaks in mid-network, and is well calibrated on-policy (Sec. 5.1).
2. **A training-free controller that turns the sensor into compute savings.** Over half of thinking tokens are removed at a small accuracy cost; at 7B the controller dominates compute-matched static budgeting (Sec. 5.2).
3. **Zero-shot operating-point transfer across scale.** A threshold selected on 1.5B dev data applies unchanged to the 7B (Sec. 5.4).
4. **Early exit helps on hard out-of-distribution multiple choice.** On GPQA-Diamond the controller beats unconstrained reasoning at both scales, indicating that unbounded chains can actively degrade answers (Sec. 5.5).
5. **Calibration under intervention is a distinct property from on-policy calibration.** We give a quantitative demonstration that a probe with ECE ≈ 0.02 on-policy is uninformative once it drives the

policy (Sec. 5.8). This is a hazard for every probe-based controller, not only ours.

6. **Two measurement pitfalls that silently invert conclusions** (Sec. 6): answer-extraction coverage, and answer-budget asymmetry between inference engines. Correcting them changed our headline result twice.

2 Related Work

Behavioral early exit. Dynasor/Certainindex [1] periodically forces an answer and exits on agreement; DEER [2] uses trial-answer confidence; SpecExit [7] and FlashThink [8] use auxiliary models or speculative signals; a related line stops when successive answers agree [12]. These are effective but pay additional decoding per check. Our probe consumes activations already computed.

Latent-state controllers. NEAT [3] monitors exit neurons and suppresses reflection tokens; AIC [4] trains gradient-boosted trees on embeddings to choose among continue/terminate/intervene; STU-PID [5] modulates steering strength with a PID controller over a redundancy classifier; TPV [6] probes reasoning progress and steers in the same direction; CGRS [11] gates reflection suppression on model certainty at the logit level. Closed-loop, state-conditioned intervention is therefore not itself novel. Our contribution is a sensor with defined semantics and audited calibration, an evaluation of every family in a single harness under one protocol, and an explicit account of when the approach fails. We are careful about the strength of the baseline comparison: the budget families received dev sweeps, but our trial-decode implementation did not, and we qualify that comparison accordingly.

Length control by training. L1/LCPO [9], Shorter-Better, and O1-Pruner train models to be concise. These are complementary: they change the model, we change only inference, and the two could compose.

Reasoning-phase representations. Venhoff et al. [10] characterize cognitive phases (deduction, verification, backtracking) as linearly represented and steerable. We reuse that taxonomy for a second probe channel and for a steering actuator, and report a negative causal result for the actuator.

3 Method

3.1 Chunk boundaries and the control loop

We decode with a batched manual loop and evaluate the controller only at *reasoning-chunk boundaries*—double-newline delimited segments of the chain of thought, which correspond to natural planning units. At each boundary the controller reads the residual stream at a fixed probe layer, evaluates the linear probes, and emits one of EXIT, STEER, or CONTINUE. EXIT splices the end-of-thinking delimiter and decodes the answer segment greedily; CONTINUE resumes sampling.

The probe layer is kept strictly below the steering layer so that the sensor cannot read its own actuator’s edit within the same forward pass, and steering directions are orthogonalized against the probe weight vectors (post-hoc cosines $\leq 10^{-3}$ in magnitude at both scales).

3.2 Convergence labels without human annotation

The convergence target is defined operationally. For every chunk boundary t in a stored rollout we truncate the chain at t , append the end-of-thinking delimiter, and decode an answer greedily. The boundary is labeled *converged* if that forced answer matches the answer the unmodified rollout eventually produced. The label is therefore a property of the model’s own computation, requires no human annotation and no external judge, and directly matches the decision the controller must make. Parse rate of forced answers was 1.000 across all datasets and both models; labels were collected at 176,448 (1.5B) and 158,961 (7B) boundaries.

The secondary phase channel (exploration / deduction / verification / backtracking / other) is labeled by regex and audited against a Qwen2.5-32B-AWQ judge; agreement is moderate (Cohen’s κ 0.43–0.52), so phase-derived results are reported as secondary and the primary convergence labels are judge-independent by construction.

3.3 Probes

Probes are logistic regressions on the residual stream at a single layer, fit on the probe-train split only. Because a naive convergence probe could be solved by position alone (later boundaries are more likely converged), we pre-registered a position-controlled gate: the probe must exceed a position-only shallow baseline by ≥ 0.05 AUC *within every position stratum*, and reach AUC ≥ 0.75 on MATH dev, before any controller experiment was permitted to run.

3.4 Policies

- EXIT_ONLY: EXIT when the convergence probability exceeds τ for K consecutive boundaries (patience), else CONTINUE.
- STEER_ONLY: add $\alpha\hat{v}$ at the steering layer when a sustained verification/backtracking loop is detected, without exiting.
- FULL: both channels.
- NOOP: unmodified decoding (the control condition).

Baselines implemented in the same harness: STATIC_BUDGET (hard token cap), BUDGET_PROMPT (budget stated in the prompt), CONCISE_PROMPT, and TRIAL_DECODE (a Dynasor-style forced-answer agreement exit).

4 Experimental Setup

Models and data. DeepSeek-R1-Distill-Qwen-1.5B (probe layer L15, steer L21) and -7B (L18, L21), fp16. Splits are fixed at seed 0: MATH-train 500 probe-train / 200 dev, MATH-500 as untouched test; GSM8K 300/50/250; AIME’24 dev and AIME’25 test ($n=30$, post-dating the model release); GPQA-Diamond ($n=198$) test-only, with operating points inherited from MATH dev and no per-dataset tuning. Sampling is temperature 0.6, top- p 0.95, with a 16k thinking cap (24k for AIME) and a 2048-token answer budget (Sec. 6).

Protocol. All tuning uses dev data only. The test split is evaluated once, at the dev-selected operating point, chosen by a 1-SE rule (best pooled dev accuracy, then fewest tokens among points within one standard error). Headline conditions use 8 seeds at 1.5B and 4 at 7B.

Statistics. The unit of analysis is the problem; conditions share problems and per-(problem, rollout) seeds, so deltas are paired. All intervals are 95% problem-level cluster bootstraps with 10,000 resamples. The pre-registered primary endpoint is pooled MATH-500 + GSM8K ($n=750$ problems), tested hierarchically: first token superiority, then accuracy non-inferiority at a 2% margin. AIME’25 and GPQA carry descriptive intervals only and are never used for tuning.

5 Results

5.1 Convergence is linearly decodable beyond position

Table 1 reports probe quality. Both models clear the pre-registered gate comfortably. AUC peaks in mid-network ($L15/28 \approx 0.54$ depth at 1.5B; $L18/28 \approx 0.64$

Model	Layer	AUC	Position margin	ECE
1.5B	L15	0.882	0.147	0.022
7B	L18	0.854	0.246	0.055

Table 1: Convergence probe quality at the selected layer. “Position margin” is the minimum AUC advantage over a position-only baseline across position strata; the pre-registered gate required ≥ 0.05 .

Pooled MATH-500 + GSM8K	Acc. [95% CI]	Tokens
<i>1.5B</i>		
EXIT_ONLY ($\tau=0.7, K=2$)	0.817 [0.796, 0.836]	1,773
NOOP (same engine)	0.856 [0.835, 0.876]	3,722
STATIC_BUDGET	0.803 [0.780, 0.825]	1,578
<i>7B (transferred point)</i>		
EXIT_ONLY	0.863 [0.843, 0.882]	1,093
NOOP (same engine)	0.936 [0.917, 0.953]	2,622
STATIC_BUDGET	0.849 [0.826, 0.870]	1,499

Table 2: Pre-registered primary endpoint. Paired token reduction is $-1,949$ $[-2,106, -1,798]$ at 1.5B and $-1,529$ $[-1,699, -1,368]$ at 7B; paired accuracy change is -0.039 $[-0.054, -0.024]$ and -0.073 $[-0.091, -0.056]$.

at 7B), and—importantly—the margin over the position-only baseline *grows* with scale ($0.147 \rightarrow 0.246$), which is the opposite of what a position artifact would predict. On-policy calibration is excellent at 1.5B (ECE 0.022) and moderate at 7B (0.055). The phase channel reaches macro AUC 0.938 / 0.937.

5.2 Main efficiency result

Table 2 reports the pre-registered primary endpoint. The controller removes 52.4% of thinking tokens at 1.5B and 58.3% at 7B, with token superiority significant by a wide margin at both scales. The accompanying accuracy change is -0.039 $[-0.054, -0.024]$ and -0.073 $[-0.091, -0.056]$: small in absolute terms, but resolvable at this sample size and larger than the 2% non-inferiority margin we pre-registered. We therefore describe the method as trading a small, measurable accuracy cost for a large compute saving, rather than as accuracy-preserving.

Against a compute-matched baseline. The comparison that matters for a practitioner is not against unconstrained decoding but against simply capping the budget. Here the result is scale-dependent and, at 7B, clean: the controller uses 27.1% fewer tokens than STATIC_BUDGET (-406 $[-499, -301]$) at equal-or-better accuracy ($+0.014$ $[+0.000, +0.029]$)—it dominates on both axes. At 1.5B it does not dominate: it buys $+0.014$ $[+0.001, +0.028]$ accuracy while spending 12% more tokens. The natural reading is that a learned exit rule pays

1.5B, test	MATH-500	GSM8K	GPQA-D	AIME'25
NOOP	.833 4653	.877 2082	.240 8027	.225 14604
EXIT_ONLY	.807 2312	.836 694	.323 6538	.237 11492
STATIC_BUDGET	.774 1726	.861 1281	.316 1923	.146 2033
BUDGET_PROMPT	.832 4514	.867 1916	.253 8015	.258 14069
CONCISE_PROMPT	.832 3447	.765 344	.296 6693	.254 14090
TRIAL_DECODE	.493 317	.584 301	.135 376	.042 325

Table 3: 1.5B test results, 8 seeds. Upper number is accuracy, lower (small) is mean think tokens. AIME'25 ($n=30$) and GPQA ($n=198$) are descriptive anchors.

off once the model is strong enough that the right stopping point genuinely varies across problems; on a weaker model, where most chains run to a similar length, a fixed cap captures most of the available benefit.

5.3 Per-dataset behavior

Table 3 gives the full test grid at 1.5B. Two patterns hold at both scales. The accuracy cost tracks how much genuine search a problem requires: it is negligible on GSM8K, moderate on MATH-500, and largest on AIME. Among the prompt-level baselines, BUDGET_PROMPT barely restrains generation at all (4,514 tokens on MATH-500 against NOOP's 4,653), so it is accuracy-preserving but not an efficiency method; CONCISE_PROMPT restrains GSM8K aggressively but not MATH.

A caveat on the trial-decode baseline. Our TRIAL_DECODE row is far below the near-lossless numbers reported by DEER [2] and Dynasor [1], and we do not present it as a faithful evaluation of that family. Its exit rule has free parameters (agreement count, minimum chunks before probing) which we ran at a single default configuration rather than sweeping, whereas EXIT_ONLY received nine dev configurations and the budget families four each. The resulting operating point is far more aggressive than any we selected for ourselves (317 vs 2,312 mean think tokens), which is sufficient to explain the gap. We report the row for completeness and explicitly do not draw a comparative conclusion from it; a properly tuned head-to-head against behavioral exit remains open work.

5.4 Operating points transfer across scale

The 7B results in Table 2 use the threshold selected on 1.5B dev data, with *no* retuning—no 7B sweep was run.

It transfers usefully: the controller remains far cheaper than unconstrained decoding and dominates static budgeting. It also transfers imperfectly, and informatively: the accuracy cost roughly doubles ($-0.039 \rightarrow -0.073$) for a comparable token saving, i.e. a threshold calibrated on a weaker model exits slightly too eagerly for a stronger one. This is consistent with the 7B probe's higher ECE (0.055 vs 0.022) and suggests threshold transfer should be accompanied by a light recalibration when the target model's probe is less well calibrated.

5.5 Early exit improves hard OOD multiple choice

On GPQA-Diamond—held out entirely from tuning, and a different task format (multiple choice, natural science)—the controller *exceeds* unconstrained reasoning at both scales. Comparing within engine, accuracy is 0.323 vs 0.249 at 1.5B (+0.074) and 0.433 vs 0.384 at 7B (+0.049), while using 16% and 60% fewer thinking tokens respectively. STATIC_BUDGET also improves over NOOP here (0.316, 0.405), so the honest statement is that restraint of any kind helps on this benchmark rather than that our controller is uniquely responsible. The finding itself is the point: on hard multiple choice, long unconstrained chains do not merely waste compute, they degrade answers—the model reasons itself away from an initially correct choice. A convergence-aware exit captures that benefit while remaining adaptive.

5.6 The steering channel fails

We pre-registered an acceptance gate for the steering actuator: it ships only if the upper confidence bound on the paired accuracy drop is below 2% on at least 200 paired dev rollouts. It fails by an order of magnitude. At $\alpha=3$ over 250 paired problems (1000 rollouts), steering costs -0.215 [$-0.251, -0.180$] accuracy and, far from saving compute, *adds* +3,583 [+3,131, +4,063] thinking tokens. A pre-registered framing gate (steering must add $\geq 5\%$ of exit-only's savings) therefore resolved the paper to be exit-led, in writing, before test results were examined.

A regex screen on the phase rates shows the intervention moving in the wrong direction: positive α *lowered* the backtracking rate (0.163 vs 0.231, $\Delta = -0.068$ [$-0.074, -0.062$]) rather than suppressing it as intended. We report this as a negative result with a mechanism (Sec. 5.8) rather than tuning until it passes: the directions are real and interpretable (Sec. 5.7), but applying them open-loop over long horizons destabilizes the chain.

5.7 What the directions encode

A logit-lens read of each direction is informative and, usefully, disagrees across channels. The phase directions decode to exactly the concepts they are named for: the backtracking direction most promotes *but, However*,

Claimed p bin	n	Mean claimed p	Realized acc.
[0.70, 0.80)	230	0.755	0.670
[0.80, 0.90)	281	0.852	0.712
[0.90, 0.95)	206	0.925	0.694
[0.95, 1.00]	233	0.971	0.691

Table 4: Closed-loop calibration audit, 1.5B at the selected operating point. The probe is well calibrated on-policy (ECE 0.022) but its asserted confidence is uninformative once it drives exits.

But, perhaps—including the corresponding Chinese discourse connectives, indicating a language-general feature rather than a token pattern—and the verification direction promotes *checking*, *check*, *verification*, *confirming*. The convergence direction does not admit such a reading: apart from the end-of-sequence token, which is exactly what one would want, its top promotions are not semantically coherent. We take this as evidence that the phase channel is a genuine interpretable feature while the convergence probe, though predictive and useful, is better described as a decodable correlate than as a clean monosemantic “I am finished” direction.

5.8 On-policy calibration does not imply calibration under control

This is, we believe, the most transferable finding in the paper. The convergence probe is well calibrated on stored rollouts (ECE 0.022). Once it *drives* the policy, that calibration disappears. Table 4 bins exited rollouts by the confidence the probe asserted at the moment of exit, and reports realized accuracy in each bin. Realized accuracy is essentially flat at ≈ 0.69 across the whole range from 0.75 to 0.97: the probe’s confidence, once it is in the loop, carries almost no information about whether the answer is actually right.

The mechanism is distribution shift of a specific and avoidable kind. The probe is trained on states produced by *unintervened* decoding, and then queried on states produced by its own interventions—a control problem structurally identical to the one imitation learning addresses with iterative on-policy data collection. This bounds what threshold tuning can achieve: our dev sweep over $\tau \in \{0.7, 0.8, 0.9, 0.95\}$ and $K \in \{1, 2\}$ is essentially flat in accuracy, because raising the bar buys tokens rather than correctness when the bar itself is uninformative. The indicated fix is not a better threshold but a probe retrained on the states it induces. We regard this as a caution for the broader class of probe-driven controllers: reporting on-policy AUC and ECE, as is standard, does not establish that a sensor will behave when it is put in charge.

6 Two Measurement Pitfalls

We report these because both are invisible in aggregate metrics, both inverted our conclusions, and neither is specific to our system.

Answer-extraction coverage. Our initial extractor accepted `\boxed{}` and “the answer is X ” but not the “**Answer:** X ” form, which these models emit in roughly 20–25% of mathematical completions and about 60% on GPQA. Those rollouts were scored incorrect. The defect was caught only because two GPQA numbers fell below the 0.25 chance floor of a four-way multiple choice—an impossibility that a purely relative comparison would have hidden. All 26,926 stored rollouts were re-graded offline from stored text, with no regeneration.

Answer-budget asymmetry across engines. Our controller ran in a HuggingFace decode loop that enforced a 512-token answer cap; the vLLM baselines received thinking and answer budgets jointly and were effectively uncapped. This truncated exactly the verbose conditions—including the NOOP control—and manifested as an apparent 0.16 “cross-engine accuracy gap” that we initially, and wrongly, attributed to the inference engine. Raising the answer budget to 2048 tokens (above the observed p99 answer length) eliminated it: the two engines now agree to within 0.013 on every dataset at both scales. Any comparison that mixes inference engines, or that caps generation per-segment rather than per-sequence, should verify this explicitly.

7 Limitations

The operating point was selected on dev data collected under the 512-token answer budget. We verified the selected point at the corrected budget and estimated the full grid’s behavior there; the 1-SE rule re-selects the same point under both conservative and generous assumptions, so we do not believe selection is a confound, but we did not re-run the entire sweep. The 7B within-engine control uses a single seed (the 7B cross-engine control uses four). The TRIAL_DECODE baseline was run at one default configuration rather than swept, so our numbers understate that family and we draw no comparative conclusion from them. AIME’25 ($n=30$) and GPQA ($n=198$) are descriptive anchors and carry no inferential claims. Phase labels have moderate judge agreement (κ 0.43–0.52), so phase results are secondary throughout. All results are on the DeepSeek-R1-Distill-Qwen family; transfer to other reasoning model families is untested.

8 Conclusion

Whether a reasoning model has finished thinking is written legibly in its residual stream, is readable with a single matrix–vector product on activations already computed, and is stable enough that a threshold chosen on a 1.5B model transfers to a 7B one without retuning. Turning that signal into a controller removes more than half of all thinking tokens for a small accuracy cost, dominates compute-matched static budgeting at 7B, and improves accuracy outright on hard out-of-distribution multiple choice.

The limits are as informative as the gains. A steering actuator built from the same representations fails decisively despite those representations being individually interpretable, and the convergence probe—well calibrated on stored rollouts—turns out to be uninformative once it is the thing choosing when to stop. We think the most useful thing this paper offers the next system is that second result: a sensor’s on-policy calibration is not evidence about its behavior in the loop, and the natural remedy is to train it on the states its own decisions create.

References

- [1] Fu, Y.; et al. Efficiently Serving LLM Reasoning Programs with Certainindex. arXiv:2412.20993, 2024.
- [2] Yang, C.; et al. Dynamic Early Exit in Reasoning Models. arXiv:2504.15895, 2025.
- [3] NEAT: Neuron-level Early-exit And Termination for reasoning models. arXiv:2602.02010, 2026.
- [4] Adaptive Inference-time Control for Reasoning Models. NeurIPS Workshop, 2025.
- [5] STU-PID: Steering Token Usage via PID Control. arXiv:2506.18831, 2025.
- [6] Overclocking LLM Reasoning: Thinking Progress Vectors. arXiv:2506.07240, 2025.
- [7] SpecExit: Speculative Early Exit for Reasoning Models. arXiv:2509.24248, 2025.
- [8] FlashThink: An Early Exit Method for Efficient Reasoning. arXiv:2505.13949, 2025.
- [9] Aggarwal, P.; and Welleck, S. L1: Controlling How Long a Reasoning Model Thinks with Reinforcement Learning. arXiv:2503.04697, 2025.
- [10] Venhoff, C.; et al. Understanding Reasoning in Thinking Language Models via Steering Vectors. arXiv:2506.18167, 2025.
- [11] Certainty-Gated Reflection Suppression. arXiv:2508.05337, 2025.
- [12] Answer Convergence as a Signal for Early Stopping. arXiv:2506.02536, 2025.